

DUDAS FRONTEND

1. Diferencia entre:

```
const [orders, setOrders] = useState({}) // llaves (para inicializar un solo objeto)
const [orders, setOrders] = useState([]) // corchetes (para inicializar un conjunto de ellos)
const [ordersAnalytics, setOrdersAnalytics] = useState(null)
```

2. Importa el orden de los fetch en el useEffect?

Depende de lo que queramos hacer, pero hacer lo más lógico

Y cómo sé cuando tengo que lanzar el useEffect (el corchete final)??

Cuando queramos refrescar la pantalla. Por ejemplo, si el usuario necesita hacer login para acceder a esa pantalla, abría que poner en los [] del final, [loggedInUser] Si en la ruta del backend se comprueba en el Get/Post/Patch... loggedInUser, entonces en el frontend, en el useEffect correspondiente hay que hacer un loggedInUser también?

Es verdad, que tiene sentido y posiblemente lo haya, pero no tomar backend como referencia, mejor razonarlo del enunciado

Qué significa poner [route] al final?

Ese route hace referencia a los parámetros entre llaves de la ruta, por ejemplo, \${id}

3. Qué diferencia hay entre type: 'danger' y type: 'error' (en el showMessage)?? Es relevante poner uno u otro al lanzar un error en el catch?
Es irrelevante, no baja nota por poner type: 'error'

```
async function fetchAddresses () {
  try {
    const data = await getAddresses()
    setAddresses(data)
  } catch (error) {
    showMessage({
      message: 'Error fetching addresses',
      description: error.toString(),
      type: 'danger',
      style: GlobalStyles.flashStyle,
      titleStyle: GlobalStyles.flashTextStyle
    })
  }
}
```

4. En la ventana de Edit (ej: EditOrderScreen.js) al inicializar los valores en el useState se pueden inicializar siempre como '' (cadenas vacías) o si fuera otro tipo de datos se podría inicializar diferente? (ej: price: 0)

```
const [initialValues, setInitialValues] = useState({ address: '', price: '' })
```

Relacionada con esta duda, entonces todos los valores que se vayan a poner en el formik hay que tratarlos como cadenas de texto? (valor que se pone en el useEffect dentro de setInitialValues)

```
price: fetchedOrder.price.toString()
```

Lo mejor es inicializar como null los datos, antes que poner 0 o una cadena vacía. Y para los datos que son números (ej: price), se inicializa como null, y en el use effect se pondría: price: fetchedOrder.price

5. Al hacer la función POST y UPDATE, ambos tienen como initialValues valores null / "" (cadenas vacías)? No tendría sentido que en el update aparezcan los valores del componente actual?

Poner siempre como initialValues el valor null en los parámetros, excepto si hay un id en la ruta que sea necesario ponerlo, como es el caso de las categoría de restaurante, que viene ya en la ruta, y se lo ponemos directamente en los valores iniciales

6. En el examen de ProductCategories, en CreateProductCategoryScreen cómo sé a dónde redirigir la pantalla cuando se crea la categoría de producto, porque el enunciado no dice nada, y la solución lo hace a RestaurantDetailScreen y yo tenía puesto ProductCategoriesScreen (que es lo que tenía para mí más sentido)

Si el enunciado no dice nada, lo más lógico es volver a la ventana directamente anterior

Se podría usar navigation.goBack() en lugar de navigation.navigate(...) ?

No, dado que navigation.goBack() no permite poner el parámetro { dirty: true }

Y el parámetro { dirty: true } sirve para refrescar la pantalla a la que renderizamos?

Sí

7. En los validation es obligatorio poner el mensaje?

Es lo más lógico y recomendable, estás demostrando que sabes lo que estás haciendo

8. Los initialValues cuando vamos a hacer un formulario, son null, "" (cadenas vacías) o son los que aparecen en la imagen (en este caso: 41700, Ej: Sevilla...)

InitialValues siempre null

Y si hubiera que poner los de la imagen, cómo los pongo??

Usando placeholders en los InputItems:

```
export default function AddressForm() {
```

```
  return (
```

```
    <Formik
```

```
      initialValues={initialValues}
```

```
      validationSchema={validationSchema}
```

```
      onSubmit={createAddress}
```

```
    >
```

```
    <{{ handleSubmit, isValid, values, setFieldValue }} => (
```

```
      <ScrollView contentContainerStyle={styles.scrollContainer}>
```

```

<KeyboardAvoidingView
  behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
  style={styles.keyboardView}
>
  <View style={styles.container}>
    <TextSemibold textStyle={styles.title}>
      Nueva dirección
    </TextSemibold>

    <InputItem label="Alias" name="alias" placeholder="Casa, Trabajo..." />
    <InputItem label="Calle" name="street" placeholder="Ej: Mejos 1" />
    <InputItem label="Ciudad" name="city" placeholder="Ej: Dos Hermanas" />
    <InputItem label="Provincia" name="province" placeholder="Ej: Sevilla" />
    <InputItem label="Código postal" name="zipCode" placeholder="41700"
keyboardType="numeric" />

    <View style={styles.toggleContainer}>
      <TextSemibold textStyle={styles.toggleLabel}>
        Dirección predeterminada
      </TextSemibold>
      <Switch
        value={values.isDefault}
        onValueChange={val => setFieldValue('isDefault', val)}
        thumbColor={values.isDefault ? brandPrimary : brandSuccessDisabled}
        trackColor={{ true: brandPrimaryTap, false: '#ccc' }}
      />
    </View>

    ...
    ...
    ...

```

AddressDetail

Nueva dirección

Alias

Casa, Trabajo...

El alias es obligatorio

Calle

Ej: Mejos 1

La calle es obligatoria

Ciudad

Ej: Dos Hermanas

La ciudad es obligatoria

Provincia

Ej: Sevilla

Código postal

41700

El código postal es obligatorio

Dirección predeterminada

Guardar dirección

Restaurants

My Orders

Profile

- En el examen tendré que registrar las pantallas en el [Stack.js](#) o vendrán ya registradas?
Las tienes que registrar tú incluso si no te dicen nada. Mira el [Stack.js](#) y si no están las añades
- Los verbos que cojo para hacer los endpoints de dónde los cojo?
- cómo añadir un valor a un `DropDownPicker`, por ejemplo, al editar un producto tiene que rellenar el campo horario, y tiene que seleccionar uno de los horarios del restaurante. Si tuviera que añadir la opción 'Not scheduled' cómo sería?
`<DropDownPicker`
- `open={open}`

```

13.         value={values.scheduleId}
14.         items={schedules}
15.         setOpen={setOpen}
16.         onSelectItem={item => {
17.             setFieldValue('scheduleId', item.value)
18.         }}
19.         setItems={setSchedules}
20.         placeholder="Not Scheduled"
21.         containerStyle={{ height: 40, marginBottom: 10 }}
22.         style={{ backgroundColor:
    GlobalStyles.brandBackground }}
23.         dropDownStyle={{ backgroundColor: '#fafafa' }}
24.     />
25.     <ErrorMessage name={'productCategoryId'}
    render={msg => <TextError>{msg}</TextError> }/>
26.

```

APUNTES

Símbolo del euro: €

UseEffect de referencia para traer datos para que aparezcan en el update:

```

useEffect(() => {
    async function fetchScheduleDetail () {
        try {
            const fetchedSchedule = await
getRestaurantSchedule(route.params.restaurantId,
route.params.scheduleId)
            setSchedule(fetchedSchedule)
            setInitialScheduleValues(buildInitialValues(fetchedSchedule,
initialScheduleValues))
        } catch (error) {
            showMessage({
                message: `There was an error while retrieving schedule
details (id ${route.params.id}). ${error} `,
                type: 'error',
                style: GlobalStyles.flashStyle,
                titleStyle: GlobalStyles.flashTextStyle
            })
        }
    }
    fetchScheduleDetail()
}, [route])

```

Respecto a los props de View y Text:

```
<View style={{ flexDirection: 'row', justifyContent: 'space-between' }}>
```

- Elementos en una fila donde se empuja un elemento a la izquierda y otro a la derecha.

```
<TextRegular textStyle={{ color: GlobalStyles.brandPrimary }}>Not scheduled</TextRegular>
```

- Texto en color rojo

```
<TextRegular textStyle={{ color: GlobalStyles.brandGreen }}>{item.schedule.startTime} - {item.schedule.endTime}</TextRegular>
```

- Texto en verde

```
<View style={{ flex: 1, flexDirection: 'row', justifyContent: 'flex-end' }}>  
  <TextSemiBold textStyle={{ color: item.products.length === 0 ?  
GlobalStyles.brandPrimary : GlobalStyles.brandSecondary  
}}>{item.products.length} products associated</TextSemiBold>  
</View>
```

- flex: 1 (ocupe todo el espacio disponible en su eje);
flexDirection: 'row' (todos los elementos en una fila)
justifyContent: 'flex-end' (todo el contenido a la derecha de la pantalla)

Para validationSchema:

- Hay que hacer validaciones para los campos que salen en pantalla, y para restaurantId en el caso de que el elemento depende del restaurante(ej: productCategory tiene que pertenecer a un restaurante, por lo que hay que hacerle la validación):
const initialProductValues = { name: null, restaurantId: route.params.id }
- Para integer:
 - .number() : pasa el dato a número, o lo intenta
 - .morethan(0) : el valor introducido debe ser **estrictamente mayor que 0**
 - .min(0) : El valor debe ser mayor o igual a 0
 - .matches(/^[0-9]{5}\$/, 'Código postal debe tener 5 dígitos') : el código postal tiene que tener 5 dígitos

Para EditScreen:

- Para Formik:
 - <InputItem name="price" label="Price:" keyboardType="numeric" /> : el keyboard hace que cuando el usuario pulse en el campo se le abra el teclado de sólo números

Para Screen:

- renderHeader: se define lo que es global, lo que aparecerá arriba del todo siempre

- renderRestaurant, renderCategory : se define lo que es individual, para cada restaurante/categoría qué tiene que aparecer
`<View style={styles.categoryRow}>` : esta cómo primera View indicando la estructura de los datos
`<TextRegular style={styles.categoryName}>{item.name}</TextRegular>` : de esta forma conseguimos que aparezca el nombre de cada categoría
- renderEmptyCategoriesList: en el caso de que no haya categorías, qué aparece en la pantalla (mensaje amable diciendo que no se han encontrado categorías)

Para CreateScreen:

- Sólo hay que hacer useEffect si fuéramos a cargar datos en la pantalla. Es decir, que si queremos crear una nueva categoría de producto, no hay que hacer useEffect porque los campos están vacíos al cargar la pantalla.
- Antes de los InputItem, poner lo siguiente si aparece un título en negrita arriba del formulario:

```
<TextSemibold textStyle={styles.title}>
```



- Los InputItem tienen el name, que debe coincidir con los nombres definidos en initialValues, y tienen el label que es lo que aparecerá en el formulario:

```
<InputItem name="shippingCosts" label="Shipping costs:" />
```

- `<InputItem name="email" label="Email:" />`

- Si hubiera que poner un switch para poner un valor por defecto, el código sería el siguiente (el isDefault se define en los initialValues definidos arriba):

```
<View style={styles.toggleContainer}>
  <TextSemibold textStyle={styles.toggleLabel}>
    Dirección predeterminada
  </TextSemibold>
  <Switch
    value={values.isDefault}
    onValueChange={val => setFieldValue('isDefault', val)}
    thumbColor={values.isDefault ? brandPrimary : brandSuccessDisabled}
    trackColor={{ true: brandPrimaryTap, false: '#ccc' }}
  />
</View>
```

- Poner justo antes del último Guardar Pressable :
`{backendErrors && backendErrors.map((error, index) => <TextError key={index}>{error.param}-{error.msg}</TextError>)}`

- DropDownPicker:

1. Un estado para almacenar las categorías de restaurantes:

```
const [restaurantCategories, setRestaurantCategories] = useState([])
```

2. Un estado booleano para establecer si la lista de opciones DropDownPicker es visible o no:

```
const [open, setOpen] = useState(false)
```

3. Un `useEffect` gancho para recuperar las categorías de restaurantes desde el backend

- En el Formik:

`handleSubmit`: es la función que activa la validación. Debe llamarse cuando el usuario pulsa el botón de envío.

`values`: es el conjunto de elementos que representa el estado del formulario.

`setFieldValue` En ocasiones, será necesario gestionar manualmente el almacenamiento de los valores de los campos. Esta función recibe como primer parámetro el nombre del campo y como segundo parámetro el valor correspondiente. Será necesaria para campos no estándar, `InputItems` como `Dropdown/select` los controles de entrada.

Para `src/api componenteEndpoints.js`:

- Aquí se definen las llamadas de la api, que tiene que hacer al backend
- Los nombres de las rutas no se inventan, se sacan del backend `src/routes` y buscas el componente del que estás haciendo el endpoint. Además, saldrán las variables que cambian (el id), para ponerlo como `${variable}`
- Parámetros
 - Pedir una lista completa (No necesita parámetros)
 - Pedir un elemento específico (Necesita un ID)
 - Crear o Editar algo (Necesita datos)

```
function createOrder (cart) {  
  return post('orders', cart)  
}
```

Iconos más usados (MaterialCommunityIcons)

Acción	name
Borrar	'delete'
Editar	'pencil'
Guardar	'content-save'
Avanzar estado	'skip-next'
Ver detalle	'eye'
Estrella llena	'star'
Estrella vacía	'star-outline'

Añadir	'plus-circle'
Volver	'arrow-left'

ÓSCAR:

render sirve para la forma en la que se ve el product, flatlist es lo que lo manda (tiene siempre la misma estructura). Cosas importantes de detailscreen:

```
View style={styles.actionButtonsContainer}>
  <View style={[{ flexDirection: 'row', justifyContent: 'space-between' }]}>
    {item.schedule
      ? <View style={[{ flexDirection: 'row' }]}>
        <MaterialCommunityIcons name='timetable' color={GlobalStyles.brandGreen} size={20} />
        <TextRegular textStyle={[{ color: GlobalStyles.brandGreen }]}>{item.schedule.startTime} -
      </View>
      : <View style={[{ flexDirection: 'row' }]}>
        <MaterialCommunityIcons name='timetable' color={GlobalStyles.brandPrimary} size={20} />
        <TextRegular textStyle={[{ color: GlobalStyles.brandPrimary }]}>Not Scheduled</TextRegular
      </View>
    }
  </View>
```

2 primeras líneas de estilo, a partir del <View, lo que haya dentro se mostrara por pantalla, MaterialCommunityIcons es para el icono del horario y el textRegular es lo que se ve escrito por pantalla, tanto en el caso de que exista schedule como en el que no

renderSchedules: Primero creamos el texto, le damos el formato necesario: <View style={[{ flex: 1, flexDirection: 'row', justifyContent: 'center' }]}> y ya despues añadimos las fotos necesarias, los botones y demas

Para eliminar cualquier cosa hay que hacer un state inicializado a null, para hacer la confirmacion de que quieres borrar el producto, horario o lo que necesites

flatlist, para ver info, formik para modificarla o añadir info
 delete, get-> Flatlist (no se usa backend errors)
 post, edit-> Formik (se usa backend errors)